# I concetti del Modello Relazionale

## Parte 3 – Algebra relazionale

Prof. Francesco Dicara
I.I.S.S. Galileo Galilei – Ostiglia (MN)

# Concetti chiave: insiemi

Un **insieme** è una raccolta di elementi **distinti** (senza ripetizioni e senza ordine).

$$A = \{1, 2, 3\}, \qquad B = \{3, 4, 5\}$$

- **Unione** $A \cup B$: tutti gli elementi che stanno in $A$ **o** in $B$.
  $A \cup B = \{1, 2, 3, 4, 5\}$
- **Intersezione** $A \cap B$: gli elementi comuni ad $A$ **e** a $B$.
  $A \cap B = \{3\}$
- **Differenza** $A - B$: gli elementi di $A$ che **non** stanno in $B$.
  $A - B = \{1, 2\}$

> Nell'algebra relazionale gli elementi non sono numeri ma **righe**, e queste operazioni si applicano solo a relazioni con lo **stesso schema** (stessi attributi e domini compatibili).

# Che cos'è l'algebra relazionale?

- È un **linguaggio formale** per manipolare e interrogare i dati del modello relazionale.
- Ogni operatore prende in **input una o due relazioni** e produce **una nuova relazione** come output (*proprietà di chiusura*): gli operatori si possono quindi **comporre**.
- È un linguaggio **procedurale**: l'espressione indica **quali operazioni eseguire e in quale ordine**. SQL, invece, è **dichiarativo**: descrive *cosa* si vuole ottenere, non come calcolarlo.
- È la **base teorica di SQL**: i DBMS (MySQL, PostgreSQL, …) traducono le query SQL in espressioni di algebra relazionale e le ottimizzano.

> *MySQL* non è un linguaggio ma un **DBMS** che implementa SQL.

# Struttura di base e operatori

**Relazioni.** Una relazione è una tabella con **righe** (tuple, istanze del dato) e **colonne** (attributi, proprietà del dato).

- Le relazioni sono **insiemi**: **non contengono righe duplicate** e l'**ordine delle righe non conta**.
- In SQL le tabelle possono invece contenere duplicati (si eliminano con `DISTINCT`).

| Operatore | Simbolo | Tipo | Funzione |
|---|---|---|---|
| Selezione | σ | unario | filtra le **righe** in base a una condizione |
| Proiezione | π | unario | mantiene solo alcune **colonne** |
| Rinomina | ρ | unario | cambia il nome di attributi o relazioni |
| Unione, intersezione, differenza | ∪ ∩ − | binari | operazioni insiemistiche |
| Prodotto cartesiano | × | binario | tutte le combinazioni di righe |
| Join | ⋈ | binario | combina righe correlate da una condizione |

# Esempio: tabelle "Studenti" e "Corsi"

## Studenti

| ID Studente | Nome | Cognome | Corso |
|---|---|---|---|
| 1 | Marco | Rossi | Informatica |
| 2 | Laura | Bianchi | Matematica |
| 3 | Sara | Verdi | Informatica |

## Corsi

| ID Corso | Nome Corso | Docente |
|---|---|---|
| 101 | Informatica | Prof. Neri |
| 102 | Matematica | Prof.ssa Bianchi |

**Domanda:** come estraiamo tutti gli studenti che frequentano il corso di Informatica?

# 1. Operatore di selezione (σ)

Filtra le **tuple (righe)** di una relazione in base a una **condizione** e restituisce una **nuova relazione** con le sole righe che la soddisfano.

**Notazione:** $\sigma_{\text{condizione}}(\text{Relazione})$

- Lo **schema non cambia**: stessi attributi, quindi **stesso grado**.
- La **cardinalità** del risultato è minore o uguale a quella di partenza.

**Esempio:** trova tutti gli studenti che frequentano Informatica.

$$\sigma_{\text{Corso}=\text{'Informatica'}}(\text{Studenti})$$

| ID Studente | Nome | Cognome | Corso |
|---|---|---|---|
| 1 | Marco | Rossi | Informatica |
| 3 | Sara | Verdi | Informatica |

# 2. Operatore di proiezione (π)

Seleziona alcuni **attributi (colonne)** e restituisce una **nuova relazione** con i soli attributi indicati, **eliminando le righe duplicate**.

**Notazione:** $\pi_{\text{attributi}}(\text{Relazione})$

**Esempio 1:** nome e cognome degli studenti: $\pi_{\text{Nome, Cognome}}(\text{Studenti})$

| Nome | Cognome |
|---|---|
| Marco | Rossi |
| Laura | Bianchi |
| Sara | Verdi |

**Esempio 2 (duplicati):** $\pi_{\text{Corso}}(\text{Studenti})$ restituisce solo **2 righe**, perché "Informatica" compare una volta sola:

| Corso |
|---|
| Informatica |

# 3. Operatore di unione (∪)

Combina due relazioni con **lo stesso schema** in una nuova relazione che contiene le righe presenti in **almeno una** delle due (senza duplicati).

**Notazione:** $R \cup S$

**Esempio:** studenti iscritti al laboratorio di Robotica ($R$) o al laboratorio di Teatro ($S$).

### R (Robotica)

| ID Studente | Nome |
|---|---|
| 1 | Marco |
| 3 | Sara |

### S (Teatro)

| ID Studente | Nome |
|---|---|
| 2 | Laura |
| 3 | Sara |

# 4. Intersezione (∩) e differenza (−)

Si applicano a relazioni con lo **stesso schema**, con le relazioni $R$ e $S$ della slide precedente.

- **Intersezione** $R \cap S$: restituisce le righe **comuni** a entrambe le relazioni.
- **Differenza** $R - S$: restituisce le righe presenti in $R$ **ma non in** $S$ (**non è commutativa**: $R - S \neq S - R$).

**R ∩ S** (iscritti a Robotica **e** a Teatro)

| ID Studente | Nome |
|---|---|
| 3 | Sara |

**R − S** (iscritti a Robotica **ma non** a Teatro)

| ID Studente | Nome |
|---|---|
| 1 | Marco |

**S − R** (iscritti a Teatro ma non a Robotica)

# 5. Operatore di join (⋈)

Un **join** combina due relazioni in una nuova relazione, **accoppiando le righe** che soddisfano una condizione.

**Notazione:** $R \bowtie_{\text{condizione}} S$

**Idea di fondo.** Il join equivale a un **prodotto cartesiano** seguito da una **selezione**:

$$R \bowtie_\theta S \;=\; \sigma_\theta(R \times S)$$

- $R \times S$ ha **grado** = grado($R$) + grado($S$) e **cardinalità** = $|R| \cdot |S|$ (tutte le combinazioni).
- La condizione $\theta$ (di solito *chiave esterna = chiave primaria*) tiene solo le combinazioni sensate.

**Tipologie principali:** 5.1 **Inner join** (theta, equi, naturale) · 5.2 **Outer join** (left, right, full)

# Tabelle di riferimento per i join

`ID Corso` in `Studenti` è una **chiave esterna** verso `Corsi`. Laura non è iscritta a nessun corso (`NULL`).

### Studenti

| ID Studente | Nome | ID Corso |
|---|---|---|
| 1 | Marco | C1 |
| 2 | Laura | NULL |
| 3 | Sara | C3 |

### Corsi

| ID Corso | Docente |
|---|---|
| C1 | Prof. Neri |
| C3 | Prof.ssa Verdi |

> Nelle espressioni algebriche i nomi degli attributi possono contenere spazi. In **SQL** conviene usare `ID_Corso` (oppure i nomi tra virgolette).

## 5.1.1 Equi join

**Definizione.** Combina le righe di due relazioni in base a una **condizione di uguaglianza esplicita** tra attributi (anche con nomi diversi).

**Notazione:** $R \bowtie_{R.a=S.b} S$

$$\text{Studenti} \bowtie_{\text{Studenti.ID Corso}=\text{Corsi.ID Corso}} \text{Corsi}$$

| ID Studente | Nome | Studenti.ID Corso | Corsi.ID Corso | Docente |
|---|---|---|---|---|
| 1 | Marco | C1 | C1 | Prof. Neri |
| 3 | Sara | C3 | C3 | Prof.ssa Verdi |

> L'equi join **conserva entrambe** le colonne usate nella condizione (qui `ID Corso` compare due volte, per questo si usa il prefisso `Tabella.Attributo`). Laura **non compare**: `NULL` non è uguale a nessun valore.

## 5.1.1 Equi join: considerazioni

- **Grado** del risultato = somma dei gradi delle due relazioni:
  grado(Studenti) + grado(Corsi) = 3 + 2 = **5**
- **Cardinalità** del risultato: dipende dalla condizione e **può andare da 0 a** $|R| \cdot |S|$. Nel caso tipico di un join *chiave esterna = chiave primaria*, ogni riga della tabella con la chiave esterna trova **al massimo una** corrispondenza, quindi il risultato ha **al massimo** tante righe quante ne ha quella tabella.
  Qui: **2** (le righe che trovano corrispondenza).

L'equi join si usa quando bisogna **confrontare attributi**, anche con **nomi diversi**, ed è quindi molto versatile. In combinazione con selezione e proiezione permette di ottenere un **grado finale inferiore**.

## 5.1.2 Join naturale

**Definizione.** Combina le righe di due relazioni in base all'uguaglianza di **tutti gli attributi con lo stesso nome**. Gli attributi comuni compaiono **una sola volta** nel risultato.

**Notazione:** $R \bowtie S$ (senza condizione)

$$\text{Studenti} \bowtie \text{Corsi}$$

| ID Studente | Nome | ID Corso | Docente |
|---|---|---|---|
| 1 | Marco | C1 | Prof. Neri |
| 3 | Sara | C3 | Prof.ssa Verdi |

- **Grado** = grado(Studenti) + grado(Corsi) − attributi comuni = 3 + 2 − 1 = **4**
- **Cardinalità** = 2 (le righe che trovano corrispondenza; valgono le stesse considerazioni dell'equi join).

> **Attenzione:** se le due relazioni hanno per caso **altri attributi omonimi** (es. `Nome` in entrambe), il join naturale li userà nella condizione e potrebbe restituire risultati inattesi.

# 5.2 Outer join (left, right, full)

L'**outer join** include le righe che **non hanno corrispondenza** nell'altra tabella, riempiendo con `NULL` gli attributi mancanti. L'inner join, invece, le scarta.

| Tipo | Simbolo | Include |
|---|---|---|
| **Left outer join** | ⟕ | tutte le righe della tabella di **sinistra** |
| **Right outer join** | ⟖ | tutte le righe della tabella di **destra** |
| **Full outer join** | ⟗ | tutte le righe di **entrambe** le tabelle |

**Esempio (left outer join)** con le tabelle `Studenti` e `Corsi`:

```
Studenti LEFT JOIN Corsi ON Studenti.ID_Corso = Corsi.ID_Corso
```

| ID Studente | Nome | ID Corso | Docente |
|---|---|---|---|
| 1 | Marco | C1 | Prof. Neri |
| 2 | Laura | NULL | NULL |
| 3 | Sara | C3 | Prof.ssa Verdi |

# Esempi di outer join: tabelle di riferimento

## Dipendenti

| ID Dip | Nome | Cognome |
|---|---|---|
| 1 | Marco | Rossi |
| 2 | Laura | Bianchi |
| 3 | Sara | Verdi |

## Progetti

| ID Progetto | Nome Progetto | ID Dip |
|---|---|---|
| 101 | ProgettoAlpha | 1 |
| 102 | ProgettoBeta | 2 |
| 103 | ProgettoGamma | NULL |

> Sara non ha progetti; `ProgettoGamma` non ha dipendenti assegnati.

# 5.2.1 Left outer join

Include **tutte le righe della tabella di sinistra** (Dipendenti), anche se non trovano corrispondenza nella tabella di destra (Progetti).

```
Dipendenti LEFT JOIN Progetti ON Dipendenti.ID_Dip = Progetti.ID_Dip
```

| ID Dip | Nome | Cognome | ID Progetto | Nome Progetto |
|---|---|---|---|---|
| 1 | Marco | Rossi | 101 | ProgettoAlpha |
| 2 | Laura | Bianchi | 102 | ProgettoBeta |
| 3 | Sara | Verdi | NULL | NULL |

> Sara compare con `NULL` negli attributi di Progetti. `ProgettoGamma` **non compare**.

# 5.2.2 Right outer join

Include **tutte le righe della tabella di destra** (Progetti), anche se non trovano corrispondenza nella tabella di sinistra (Dipendenti).

```
Dipendenti RIGHT JOIN Progetti ON Dipendenti.ID_Dip = Progetti.ID_Dip
```

| ID Dip | Nome | Cognome | ID Progetto | Nome Progetto |
|---|---|---|---|---|
| 1 | Marco | Rossi | 101 | ProgettoAlpha |
| 2 | Laura | Bianchi | 102 | ProgettoBeta |
| NULL | NULL | NULL | 103 | ProgettoGamma |

> `ProgettoGamma` compare con `NULL` negli attributi di Dipendenti. Sara **non compare**.

# 5.2.3 Full outer join

Combina left e right outer join: include **tutte le righe di entrambe le tabelle**.

```
Dipendenti FULL OUTER JOIN Progetti ON Dipendenti.ID_Dip = Progetti.ID_Dip
```

| ID Dip | Nome | Cognome | ID Progetto | Nome Progetto |
|---|---|---|---|---|
| 1 | Marco | Rossi | 101 | ProgettoAlpha |
| 2 | Laura | Bianchi | 102 | ProgettoBeta |
| 3 | Sara | Verdi | NULL | NULL |
| NULL | NULL | NULL | 103 | ProgettoGamma |

> Il risultato contiene le righe dell'inner join **più** le righe senza corrispondenza di entrambe le tabelle.

# I concetti del Modello Relazionale

## Parte 4 – Esercizi di algebra relazionale

Prof. Francesco Dicara
I.I.S.S. Galileo Galilei – Ostiglia (MN)

# Come risolvere un esercizio di algebra relazionale

1. **Individua le tabelle** coinvolte e le **chiavi esterne** che le collegano.
2. **Collegale con un join** (condizione: chiave esterna = chiave primaria).
3. Applica la **selezione σ** per le condizioni sulle righe.
4. Applica la **proiezione π** per ottenere solo le colonne richieste.

> Un'espressione si legge **dall'interno verso l'esterno**: prima il join, poi σ, infine π.
> Se due tabelle hanno attributi con lo stesso nome, **qualificali**: `Tabella.Attributo`.
> Le condizioni sono applicate al risultato del join; l'ottimizzatore del DBMS può anticiparle, ma il risultato **non cambia**.

# Relazioni di riferimento (esercizi su selezione e proiezione)

## Dipendenti

| ID Dipendente | Nome | Cognome |
|---|---|---|
| 1 | Marco | Rossi |
| 2 | Laura | Bianchi |
| 3 | Sara | Verdi |
| 4 | Luca | Neri |

## Progetti

| ID Progetto | Nome Progetto | ID Dipendente |
|---|---|---|
| 101 | ProgettoAlpha | 1 |
| 102 | ProgettoBeta | 2 |
| 103 | ProgettoGamma | 3 |
| 104 | ProgettoDelta | NULL |

# Esercizio 1: selezione

**Testo:** trova tutti i dipendenti con ID maggiore di 2.

$$\sigma_{\text{ID Dipendente}>2}(\text{Dipendenti})$$

| ID Dipendente | Nome | Cognome |
|---|---|---|
| 3 | Sara | Verdi |
| 4 | Luca | Neri |

# Esercizio 2: selezione

**Testo:** trova tutti i progetti che non hanno un dipendente assegnato.

$$\sigma_{\text{ID Dipendente IS NULL}}(\text{Progetti})$$

| ID Progetto | Nome Progetto | ID Dipendente |
|---|---|---|
| 104 | ProgettoDelta | NULL |

> Per verificare l'assenza di un valore si usa `IS NULL`, **non** `= NULL`: `NULL` non è uguale a nessun valore, nemmeno a se stesso.

# Esercizio 3: selezione

**Testo:** trova tutti i progetti con ID maggiore di 102.

$$\sigma_{\text{ID Progetto}>102}(\text{Progetti})$$

| ID Progetto | Nome Progetto | ID Dipendente |
|---|---|---|
| 103 | ProgettoGamma | 3 |
| 104 | ProgettoDelta | NULL |

# Esercizio 1: proiezione

**Testo:** visualizza solo i nomi e i cognomi dei dipendenti.

$$\pi_{\text{Nome, Cognome}}(\text{Dipendenti})$$

| Nome | Cognome |
|---|---|
| Marco | Rossi |
| Laura | Bianchi |
| Sara | Verdi |
| Luca | Neri |

# Esercizio 2: proiezione

**Testo:** visualizza solo i nomi dei progetti.

$$\pi_{\text{Nome Progetto}}(\text{Progetti})$$

| Nome Progetto |
|---|
| ProgettoAlpha |
| ProgettoBeta |
| ProgettoGamma |
| ProgettoDelta |

# Esercizio 3: proiezione

**Testo:** visualizza solo gli ID dei dipendenti e i relativi cognomi.

$$\pi_{\text{ID Dipendente, Cognome}}(\text{Dipendenti})$$

| ID Dipendente | Cognome |
|---|---|
| 1 | Rossi |
| 2 | Bianchi |
| 3 | Verdi |
| 4 | Neri |

# Esercizio 1: equi join con proiezione

**Testo:** trova il nome del progetto e il nome del dipendente per tutti i progetti con un dipendente assegnato.

$$\pi_{\text{Nome Progetto, Nome}} \left( \text{Progetti} \bowtie_{\text{Progetti.ID Dipendente} = \text{Dipendenti.ID Dipendente}} \text{Dipendenti} \right)$$

| Nome Progetto | Nome |
|---|---|
| ProgettoAlpha | Marco |
| ProgettoBeta | Laura |
| ProgettoGamma | Sara |

> Gli attributi usati nella condizione ( `ID Dipendente` ) restano nella relazione intermedia ma non compaiono nel risultato dopo la proiezione. `ProgettoDelta` è escluso: il suo `ID Dipendente` è `NULL` .

## Esercizio 2: equi join con selezione

**Testo:** trova i progetti con dipendenti assegnati, ma solo quelli in cui l'ID del dipendente è maggiore di 2.

$$\sigma_{\text{Progetti.ID Dipendente}>2}\left(\text{Progetti} \bowtie_{\text{Progetti.ID Dipendente}=\text{Dipendenti.ID Dipendente}} \text{Dipendenti}\right)$$

| ID Progetto | Nome Progetto | Progetti.ID Dipendente | Dipendenti.ID Dipendente | Nome | Cognome |
|---|---|---|---|---|---|
| 103 | ProgettoGamma | 3 | 3 | Sara | Verdi |

> L'equi join **mantiene entrambe le colonne** `ID Dipendente`, che hanno lo stesso nome: per questo nella selezione serve il prefisso `Progetti.` (o `Dipendenti.`) e il risultato ha 6 colonne (grado 3 + 3).

# Esercizio 3: equi join con selezione e proiezione

**Testo:** trova i nomi dei progetti e i nomi dei dipendenti assegnati, ma solo per i progetti il cui ID è maggiore di 102.

$$\pi_{\text{Nome Progetto, Nome}}\left(\sigma_{\text{ID Progetto}>102}\left(\text{Progetti} \bowtie_{\text{Progetti.ID Dipendente}=\text{Dipendenti.ID Dipendente}} \text{Dipendenti}\right)\right)$$

| Nome Progetto | Nome |
|---|---|
| ProgettoGamma | Sara |

> L'equi join conserva gli attributi duplicati nella relazione intermedia, ma la proiezione li elimina nel risultato finale. `ProgettoDelta` (104) non ha dipendente e quindi non compare.

# Esercizio 1: join naturale con proiezione

**Testo:** trova il nome del progetto e il nome del dipendente per tutti i progetti con un dipendente assegnato.

$$\pi_{\text{Nome Progetto, Nome}}\left(\text{Progetti} \bowtie \text{Dipendenti}\right)$$

| Nome Progetto | Nome |
|---|---|
| ProgettoAlpha | Marco |
| ProgettoBeta | Laura |
| ProgettoGamma | Sara |

> Il join naturale usa l'unico attributo comune, `ID Dipendente`, ed elimina automaticamente la colonna duplicata.

# Esercizio 2: join naturale con selezione

**Testo:** trova i progetti con dipendenti assegnati, ma solo quelli in cui l'ID del dipendente è maggiore di 2.

$$\sigma_{\text{ID Dipendente}>2}\left(\text{Progetti} \bowtie \text{Dipendenti}\right)$$

| ID Progetto | Nome Progetto | ID Dipendente | Nome | Cognome |
|---|---|---|---|---|
| 103 | ProgettoGamma | 3 | Sara | Verdi |

> Nel join naturale `ID Dipendente` compare **una sola volta** (grado 3 + 3 − 1 = 5): non serve qualificarlo.

# Esercizio 3: join naturale con selezione e proiezione

**Testo:** trova i nomi dei progetti e i nomi dei dipendenti assegnati, ma solo per i progetti il cui ID è maggiore di 102.

$$\pi_{\text{Nome Progetto, Nome}}\left(\sigma_{\text{ID Progetto}>102}\left(\text{Progetti} \bowtie \text{Dipendenti}\right)\right)$$

| Nome Progetto | Nome |
|---|---|
| ProgettoGamma | Sara |

> Il join naturale elimina gli attributi comuni duplicati, riducendo il grado della relazione risultante.

# Nuove tabelle di riferimento: Clienti e Ordini

## Clienti

| ID Cliente | Nome | Città |
|---|---|---|
| 1 | Anna | Roma |
| 2 | Luca | Milano |
| 3 | Maria | Torino |
| 4 | Marco | Napoli |

## Ordini

| ID Ordine | Prodotto | ID Cliente |
|---|---|---|
| 101 | Laptop | 1 |
| 102 | Smartphone | 2 |
| 103 | Tablet | 3 |
| 104 | Monitor | NULL |

# Esercizio 1: equi join con selezione e proiezione

**Testo:** trova i nomi dei clienti e i prodotti acquistati da chi vive a "Roma".

$$\pi_{\text{Nome, Prodotto}}\left(\sigma_{\text{Città}=\text{'Roma'}}\left(\text{Clienti} \bowtie_{\text{Clienti.ID Cliente}=\text{Ordini.ID Cliente}} \text{Ordini}\right)\right)$$

| Nome | Prodotto |
|---|---|
| Anna | Laptop |

> La selezione `Città = 'Roma'` è applicata al risultato del join. Il risultato sarebbe identico filtrando prima `Clienti` (è ciò che spesso fa l'ottimizzatore, per eseguire join su meno righe).

# Esercizio 2: equi join con selezione e proiezione

**Testo:** trova i nomi dei clienti e i prodotti acquistati, ma solo per gli ordini con ID maggiore di 102.

$$\pi_{\text{Nome, Prodotto}}\left(\sigma_{\text{ID Ordine}>102}\left(\text{Clienti} \bowtie_{\text{Clienti.ID Cliente}=\text{Ordini.ID Cliente}} \text{Ordini}\right)\right)$$

| Nome | Prodotto |
|---|---|
| Maria | Tablet |

> Gli ordini con ID > 102 sono il 103 e il 104. Il 104 (Monitor) non ha cliente ( `NULL` ), quindi **non trova corrispondenza** nel join e non compare.

## Esercizio 3: equi join con selezione e proiezione

**Testo:** trova i nomi dei clienti e i prodotti acquistati, ma solo per i clienti con ID maggiore di 2.

$$\pi_{\text{Nome, Prodotto}}\left(\sigma_{\text{Clienti.ID Cliente}>2}\left(\text{Clienti} \bowtie_{\text{Clienti.ID Cliente}=\text{Ordini.ID Cliente}} \text{Ordini}\right)\right)$$

| Nome | Prodotto |
|---|---|
| Maria | Tablet |

> I clienti con ID > 2 sono Maria (3) e Marco (4). **Marco non ha ordini** (nessuna riga di `Ordini` ha `ID Cliente = 4`) e l'ordine "Monitor" ha `ID Cliente = NULL`: nessuno dei due compare nel join. Si qualifica `Clienti.ID Cliente` perché l'attributo è presente in entrambe le tabelle.

# Nuove tabelle di riferimento: Studenti ed Esami

## Studenti

| ID Studente | Nome | Classe |
|---|---|---|
| 1 | Marco | 5A |
| 2 | Anna | 5B |
| 3 | Luca | 4A |
| 4 | Sara | 5A |
| 5 | Maria | 4B |

## Esami

| ID Esame | Materia | ID Studente |
|---|---|---|
| 101 | Matematica | 1 |
| 102 | Inglese | 2 |
| 103 | Fisica | 1 |
| 104 | Chimica | 4 |
| 105 | Storia | 5 |

# Esercizio 1: equi join con selezione e proiezione

**Testo:** trova i nomi degli studenti e le materie d'esame per gli studenti della classe "5A".

$$\pi_{\text{Nome, Materia}}\left(\sigma_{\text{Classe}=\text{'5A'}}\left(\text{Studenti} \bowtie_{\text{Studenti.ID Studente}=\text{Esami.ID Studente}} \text{Esami}\right)\right)$$

| Nome | Materia |
|---|---|
| Marco | Matematica |
| Marco | Fisica |
| Sara | Chimica |

> `Classe = '5A'` seleziona solo gli studenti della 5A. L'equi join associa ogni studente ai propri esami tramite `ID Studente`: Marco compare **due volte** perché ha sostenuto due esami.

## Esercizio 2: equi join con selezione complessa e proiezione

**Testo:** trova i nomi degli studenti, le materie d'esame e le classi, ma solo per gli esami il cui ID è maggiore di 102 e appartengono a studenti di classi quarte.

$$\pi_{\text{Nome, Materia, Classe}}\left(\sigma_{\text{ID Esame}>102 \,\wedge\, \text{Classe LIKE '4\%'}}\left(\text{Studenti} \bowtie_{\text{Studenti.ID Studente}=\text{Esami.ID Studente}} \text{Esami}\right)\right)$$

| Nome | Materia | Classe |
|---|---|---|
| Maria | Storia | 4B |

- `ID Esame > 102`: filtra gli esami con ID maggiore di 102.
- `Classe LIKE '4%'`: seleziona le classi che **iniziano per 4** (`%` = qualsiasi sequenza di caratteri).
- `AND` (∧): **entrambe** le condizioni devono essere vere.
- Il join collega ogni studente ai propri esami tramite `ID Studente`.

## Esercizio 3: equi join con selezione e proiezione multipla

**Testo:** trova i nomi degli studenti e le materie d'esame, ma solo per gli esami di "Matematica" o "Fisica".

$$\pi_{\text{Nome, Materia}}\left(\sigma_{\text{Materia='Matematica'} \vee \text{Materia='Fisica'}}\left(\text{Studenti} \bowtie_{\text{Studenti.ID Studente}=\text{Esami.ID Studente}} \text{Esami}\right)\right)$$

| Nome | Materia |
|---|---|
| Marco | Matematica |
| Marco | Fisica |

- `Materia = 'Matematica' OR Materia = 'Fisica'`: l'`OR` ($\vee$) seleziona le righe in cui **almeno una** delle due condizioni è vera.
- Il join associa ogni studente ai propri esami tramite `ID Studente`.

## Esercizio 4: equi join con selezione complessa e proiezione

**Testo:** trova i nomi degli studenti, le materie d'esame e le classi, ma solo per gli esami di "Matematica" o "Fisica" sostenuti da studenti della classe "5A" o "4B", il cui ID esame è **dispari**.

$$\pi_{\text{Nome, Materia, Classe}}\Big(\sigma_{c}\big(\text{Studenti} \bowtie_{\text{Studenti.ID Studente}=\text{Esami.ID Studente}} \text{Esami}\big)\Big)$$

$$c: \ (\text{Materia} = \text{'Matematica'} \lor \text{Materia} = \text{'Fisica'}) \land (\text{Classe} = \text{'5A'} \lor \text{Classe} = \text{'4B'}) \land \text{ID Esame} \bmod 2 = 1$$

| Nome | Materia | Classe |
|---|---|---|
| Marco | Matematica | 5A |
| Marco | Fisica | 5A |

- Le prime due condizioni filtrano materia e classe; `ID Esame mod 2 = 1` (in SQL: `% 2 = 1`) tiene solo gli ID dispari (101 e 103).
- Le parentesi stabiliscono l'**ordine di valutazione** di `AND` e `OR`.

# Tabelle di riferimento (modello logico)

Schemi delle relazioni (**PK** in grassetto, *FK* in corsivo):

- `Docenti(` **ID Docente** `, Nome, Dipartimento)`
- `Corsi(` **ID Corso** `, Nome Corso,` *ID Docente* `)`
- `Studenti(` **ID Studente** `, Nome, Classe,` *ID Corso* `)`

### Docenti

| ID Docente | Nome | Dipartimento |
|---|---|---|
| 1 | Prof. Bianchi | Matematica |
| 2 | Prof. Rossi | Fisica |
| 3 | Prof. Verdi | Informatica |

### Corsi

| ID Corso | Nome Corso | ID Docente |
|---|---|---|
| C1 | Analisi 1 | 1 |
| C2 | Fisica 1 | 2 |
| C3 | Programmazione | 3 |

# Tabella Studenti (modello logico)

### Studenti

| ID Studente | Nome | Classe | ID Corso |
|---|---|---|---|
| 1 | Giulia | 4A | C2 |
| 2 | Luca | 5B | C2 |
| 3 | Elisa | 5A | C3 |
| 4 | Marco | 5A | C4 |
| 5 | Anna | 4B | C1 |

**Chiavi primarie:** identificano in modo univoco ogni riga della tabella.
**Chiavi esterne:** collegano una tabella a un'altra riferendo le sue chiavi primarie ( `Corsi.ID Docente → Docenti` , `Studenti.ID Corso → Corsi` ).

# Esercizio 1: join e proiezione

**Testo:** trova i nomi dei docenti e i nomi dei corsi che insegnano.

$$\pi_{\text{Docenti.Nome, Nome Corso}} \left( \text{Docenti} \bowtie_{\text{Docenti.ID Docente} = \text{Corsi.ID Docente}} \text{Corsi} \right)$$

| Nome | Nome Corso |
|---|---|
| Prof. Bianchi | Analisi 1 |
| Prof. Rossi | Fisica 1 |
| Prof. Verdi | Programmazione |
| Prof. Verdi | Basi di Dati |

> L'equi join associa i docenti ai corsi tramite `ID Docente`. La proiezione restituisce solo `Nome` (docente) e `Nome Corso`. Il Prof. Verdi compare **due volte**, perché insegna due corsi.

## Esercizio 2: join, selezione e proiezione

**Testo:** trova i nomi degli studenti che frequentano corsi insegnati da docenti del dipartimento di "Fisica".

$$\pi_{\text{Studenti.Nome}}\Big(\sigma_{\text{Dipartimento}=\text{'Fisica'}}\big(\text{Studenti} \bowtie_{\text{Studenti.ID Corso}=\text{Corsi.ID Corso}} (\text{Corsi} \bowtie_{\text{Corsi.ID Docente}=\text{Docenti.ID Docente}} \text{Docenti})\big)\Big)$$

| Nome |
|---|
| Giulia |
| Luca |

- Si risolve dall'interno: `Corsi ⋈ Docenti` (corsi con il proprio docente), poi il join con `Studenti`.
- La selezione `Dipartimento = 'Fisica'` tiene il corso **C2** (Fisica 1, Prof. Rossi), frequentato da Giulia e Luca.
- Due join concatenati collegano `Studenti`, `Corsi` e `Docenti` tramite le **chiavi esterne**.

## Esercizio 3: join complesso, selezione e proiezione

**Testo:** trova i nomi degli studenti e i nomi dei corsi frequentati, ma solo per studenti di classi quinte che frequentano corsi insegnati dal dipartimento di "Informatica".

$$\pi_{\text{Studenti.Nome, Nome Corso}}\Big(\sigma_{\text{Classe LIKE '5\%'} \,\wedge\, \text{Dipartimento='Informatica'}}\big(\text{Studenti} \bowtie_{\text{Studenti.ID Corso=Corsi.ID Corso}} (\text{Corsi} \bowtie_{\text{Corsi.ID Docente=Docenti.ID Docente}} \text{Docenti})\big)\Big)$$

| Nome | Nome Corso |
|---|---|
| Elisa | Programmazione |
| Marco | Basi di Dati |

- `Classe LIKE '5%'`: filtra gli studenti delle classi quinte (Elisa 5A, Marco 5A; Luca 5B segue un corso di Fisica).
- `Dipartimento = 'Informatica'`: seleziona i corsi del Prof. Verdi.
- I join concatenano `Studenti`, `Corsi` e `Docenti` tramite le chiavi esterne; la proiezione restituisce solo i campi richiesti.

# Operazioni relazionali – Schema riassuntivo

Prof. Francesco Dicara
I.I.S.S. Galileo Galilei – Ostiglia (MN)

# Selezione (σ): condizioni numeriche

**Definizione.** La selezione è un'operazione **unaria** che restituisce le tuple di una relazione che soddisfano una **condizione**.

**Notazione:** $\sigma_{\text{condizione}}(R)$

Condizioni numeriche (operatori `=`, `>`, `<`, `>=`, `<=`, `!=`):

- Attributo $=$ Valore (es. `Età = 18`)
- Attributo $>$ Valore (es. `Età > 18`)
- Attributo $<$ Valore (es. `Salario < 2000`)
- Attributo $\geq$ Valore, Attributo $\leq$ Valore

**Esempi pratici**

$$\sigma_{\text{Età}>18}(\text{Studenti}), \qquad \sigma_{\text{Salario}\leq 1500}(\text{Impiegati}), \qquad \sigma_{\text{Voto}\geq 8}(\text{Esami})$$

Restituiscono rispettivamente: gli studenti con età maggiore di 18, gli impiegati con salario fino a 1500, gli esami con voto almeno 8.

> **N.B.** Si possono usare gli operatori logici `AND`, `OR`, `NOT` per unire più condizioni nella stessa selezione.

# Selezione (σ): condizioni sulle stringhe

- Attributo `= 'Valore'` (es. `Nome = 'Mario'`)
- Attributo `!= 'Valore'` (es. `Città != 'Roma'`)
- Operatori logici: `AND`, `OR`, `NOT`
- Confronti con **stringhe parziali** (`LIKE`), dove `%` indica una sequenza qualsiasi di caratteri:
  - `LIKE 'A%'` → inizia con "A"
  - `LIKE '%z'` → finisce con "z"
  - `LIKE '%par%'` → contiene "par"

**Esempi pratici**

$$\sigma_{\text{Città}=\text{'Milano'}}(\text{Clienti}), \qquad \sigma_{\text{Nome LIKE 'A\%'}}(\text{Studenti})$$

$$\sigma_{\text{Cognome LIKE '\%ini'}}(\text{Professori}), \qquad \sigma_{\text{Email NOT LIKE '\%gmail.com'}}(\text{Utenti})$$

Restituiscono rispettivamente: i clienti di Milano, gli studenti il cui nome inizia con "A", i professori con cognome che termina in "ini" e gli utenti senza email Gmail.

# Proiezione (π)

**Definizione.** La proiezione seleziona solo **alcuni attributi** di una relazione, **eliminando eventuali duplicati**.

**Notazione:** $\pi_{\text{Attributi}}(R)$

**Esempio:**

$$\pi_{\text{Nome, Età}}(\text{Studenti})$$

Restituisce una relazione con **solo** i campi `Nome` ed `Età`.

> La selezione **filtra le righe**; la proiezione **filtra le colonne**.

# Equi join

**Definizione.** L'equi join combina due relazioni in base a una **condizione di uguaglianza** esplicita tra attributi.

**Notazione:** $R \bowtie_{R.a=S.b} S$

**Esempio:**

$$\text{Studenti} \bowtie_{\text{Studenti.Classe}=\text{Classi.ID}} \text{Classi}$$

Combina gli studenti con le classi corrispondenti, usando il campo `Classe` (chiave esterna) e `ID` (chiave primaria di `Classi`).

# Join naturale

**Definizione.** Il join naturale è un **caso particolare dell'equi join**: la condizione di uguaglianza è **implicita** su tutti gli attributi con lo stesso nome, che compaiono **una sola volta** nel risultato.

**Notazione:** $R \bowtie S$

**Esempio:**

$$\text{Studenti} \bowtie \text{Classi}$$

Combina gli studenti con le classi usando **tutti gli attributi comuni** tra le due relazioni.

# Conclusione: riepilogo delle operazioni

| Operazione | Simbolo | Funzione | In SQL |
|---|---|---|---|
| **Selezione** | σ | Filtra le **righe** (tuple) | `WHERE` |
| **Proiezione** | π | Filtra le **colonne** (attributi) | `SELECT` + `DISTINCT` |
| **Unione, intersezione, differenza** | ∪ ∩ − | Operazioni insiemistiche | `UNION`, `INTERSECT`, `EXCEPT` |
| **Equi join** | ⋈ | Combina relazioni con una condizione di uguaglianza esplicita | `JOIN … ON` |
| **Join naturale** | ⋈ | Combina relazioni sugli attributi omonimi | `NATURAL JOIN` |
| **Outer join** | ⟕ ⟖ ⟗ | Come l'equi join, ma conserva le righe senza corrispondenza | `LEFT / RIGHT / FULL JOIN` |

> L'algebra relazionale è la **base teorica di SQL**: saper scrivere un'espressione algebrica significa saper pensare una query.

# La normalizzazione nel modello relazionale

Prof. Francesco Dicara · Informatica

# Di cosa parliamo

1. Il problema: **ridondanza** e **anomalie**
2. Gli strumenti: **chiavi** e **dipendenze funzionali**
3. Le forme normali: **1FN** → **2FN** → **3FN** (+ cenno a **BCNF**)
4. L'**algoritmo di scomposizione**
5. Un esempio completo, dalla tabella unica allo **schema SQL**
6. Costi della normalizzazione e **denormalizzazione**
7. Normalizzazione e **esame di maturità** + esercizi

# Il problema: la ridondanza

> Si ha **ridondanza** quando la **stessa informazione** è memorizzata **più volte** nel database.

Perché è un problema?

- **spreco di spazio** su disco (il problema minore)
- rischio di **inconsistenza**: due copie dello stesso dato possono diventare **diverse**
- **anomalie** durante inserimento, modifica e cancellazione

> Idea chiave: **ogni fatto della realtà deve essere scritto in un solo posto.**

# Esempio guida: gli ordini di un e-commerce

Un negozio online salva tutto in **un'unica tabella** `OrdiniFlat`:

| NumOrdine | Data | CodCli | NomeCli | Città | CodProd | Descrizione | Prezzo | Qtà |
|---|---|---|---|---|---|---|---|---|
| 1001 | 03/10 | C01 | Rossi | Roma | P10 | Mouse | 15 | 2 |
| 1001 | 03/10 | C01 | Rossi | Roma | P20 | Tastiera | 30 | 1 |
| 1002 | 05/10 | C02 | Bianchi | Milano | P10 | Mouse | 15 | 1 |
| 1003 | 07/10 | C01 | Rossi | Roma | P30 | Monitor | 180 | 1 |
| 1003 | 07/10 | C01 | Rossi | Roma | P20 | Tastiera | 30 | 2 |

Chiave primaria: **{NumOrdine, CodProd}** (un ordine contiene più prodotti).

Quanti doppioni vedi? `Rossi, Roma` ×4 · `Tastiera, 30` ×2 · data dell'ordine 1001 ×2 …

# Le tre anomalie

| Anomalia | Cosa succede nell'esempio |
|---|---|
| **Aggiornamento** | Rossi si trasferisce a Torino: va modificata la città in **4 righe**. Se se ne dimentica una, il DB dice che Rossi abita in due città → **dati inconsistenti** |
| **Cancellazione** | Annullo l'ordine 1002 (unico ordine di Bianchi): perdo **anche i dati del cliente** Bianchi |
| **Inserimento** | Arriva un nuovo prodotto P40 "Webcam", mai ordinato: **non posso inserirlo**, perché `NumOrdine` fa parte della chiave e non può essere NULL |

> Tutte e tre nascono dallo stesso errore: **informazioni su "cose" diverse** (ordini, clienti, prodotti) **mescolate nella stessa tabella**.

# Che cos'è la normalizzazione

> La **normalizzazione** è un procedimento che **trasforma** (scompone) le tabelle per eliminare **ridondanza** e **anomalie**, senza perdere informazioni.

Dove si colloca nella progettazione:

**Realtà** → schema **concettuale** (E/R) → schema **logico** (tabelle) → **verifica con la normalizzazione** → schema **fisico** (SQL)

- se l'E/R è fatto bene, le tabelle risultano già (quasi) normalizzate
- la normalizzazione è il **controllo di qualità** dello schema logico
- si basa su **forme normali** di livello crescente: ognuna include le precedenti

# Gli strumenti: le chiavi

> **Chiave candidata**: attributo o insieme **minimale** di attributi che identifica **univocamente** ogni riga.

- **Chiave primaria**: la chiave candidata **scelta** dal progettista (una sola per tabella)
- **Chiave alternativa**: ogni altra chiave candidata non scelta
- **Attributo non-chiave**: attributo che **non fa parte** di nessuna chiave

Esempio: `Studenti(Matricola, CodiceFiscale, Email, Cognome, Classe)`

| Matricola | CodiceFiscale | Email | → chiavi candidate |
|---|---|---|---|
| scelta come **PK** | chiave **alternativa** | chiave **alternativa** | Cognome, Classe → **non-chiave** |

# Attenzione alla parola "minimale"

**Superchiave** = insieme di attributi che identifica le righe, anche con attributi **in più**.
**Chiave candidata** = superchiave da cui **non posso togliere nulla**.

Nella tabella `OrdiniFlat`:

| Insieme di attributi | Identifica la riga? | Minimale? | È chiave candidata? |
|---|---|---|---|
| {NumOrdine} | No (1001 compare 2 volte) | – | **No** |
| {CodProd} | No (P10 in più ordini) | – | **No** |
| {NumOrdine, CodProd} | Sì | Sì | **Sì** |
| {NumOrdine, CodProd, Qtà} | Sì | No (Qtà è superflua) | **No**, è solo superchiave |

# Gli strumenti: la dipendenza funzionale

> Si ha **dipendenza funzionale** `A → B` quando, **fissato il valore di A, il valore di B è unico**.
> Si legge: "*A determina B*" oppure "*B dipende funzionalmente da A*". A è il **determinante**.

Esempi nella realtà dell'e-commerce:

- `CodCli → NomeCli, Città` (un codice cliente = un solo cliente)
- `CodProd → Descrizione, Prezzo`
- `NumOrdine → Data, CodCli` (un ordine ha una data e un cliente)
- `{NumOrdine, CodProd} → Qtà` (la quantità dipende da *quale prodotto* in *quale ordine*)

Collegamento con le chiavi: **una chiave candidata determina tutti gli attributi della tabella**.

# Le dipendenze si ricavano dalla realtà, non dai dati

> Una dipendenza funzionale è una **regola del mondo reale**, non una coincidenza nei dati di esempio.

Nella tabella vediamo che ogni cliente ha **una sola** città. Ma:

- `Città → NomeCli`? Nei dati Roma → Rossi, Milano → Bianchi… ma è **falso**: a Roma abitano molti clienti
- `NomeCli → CodCli`? Falso: esistono **omonimi**

Quindi: per trovare le DF bisogna **conoscere il dominio** (cosa dice il testo della traccia!). È il passo **non automatizzabile** della normalizzazione.

# Dipendenze parziali e transitive

## Dipendenza parziale

Un attributo non-chiave dipende da **una parte** della chiave composta.

Chiave `{NumOrdine, CodProd}`

`CodProd → Descrizione`

Descrizione dipende solo da **metà** della chiave.

## Dipendenza transitiva

Un attributo non-chiave dipende dalla chiave **passando per** un altro attributo non-chiave.

`NumOrdine → CodCli`

`CodCli → NomeCli`

⇒ `NumOrdine → NomeCli` **transitivamente**

> **Parziale** → si elimina con la **2FN** · **Transitiva** → si elimina con la **3FN**

# Le forme normali: una scala

| Forma | Requisito (in breve) | Elimina |
|---|---|---|
| **1FN** | attributi **atomici**, niente gruppi ripetuti | attributi multivalore / composti |
| **2FN** | 1FN + ogni non-chiave dipende dall'**intera** chiave | dipendenze **parziali** |
| **3FN** | 2FN + ogni non-chiave dipende **direttamente** dalla chiave | dipendenze **transitive** |
| **BCNF** | ogni determinante è una **superchiave** | casi residui della 3FN |

Ogni livello **include** i precedenti: una tabella in 3FN è anche in 2FN e in 1FN.

Nella pratica (e all'esame) l'obiettivo è **almeno la 3FN**.

# Prima forma normale (1FN)

> Una relazione è in **1FN** se rispetta i requisiti di base del modello relazionale:

- tutte le righe hanno lo **stesso numero di colonne**
- gli attributi sono **atomici** (non scomponibili): niente sottoattributi, niente liste
- i valori di una colonna appartengono allo **stesso dominio**
- **non** esistono due righe **uguali**
- l'**ordine** delle righe (e delle colonne) è **irrilevante**

In pratica, le violazioni tipiche sono tre: **attributi composti**, **attributi multivalore**, **gruppi di colonne ripetute**.

# 1FN: esempio di violazione (biblioteca scolastica)

`Utenti(Tessera, Nome, Indirizzo, Telefoni, Libro1, Libro2, Libro3)`

| Tessera | Nome | Indirizzo | Telefoni | Libro1 | Libro2 | Libro3 |
|---|---|---|---|---|---|---|
| T01 | Neri | Via Po 3, 10100 Torino | 333-111, 011-222 | L07 | L12 | |
| T02 | Galli | P.za Dante 1, 80100 Napoli | 347-555 | L03 | | |

- **Indirizzo** è **composto** (via, CAP, città): non posso cercare "utenti di Torino" in modo affidabile
- **Telefoni** è **multivalore**: due numeri nella stessa cella
- **Libro1..3** è un **gruppo ripetuto**: e se un utente prende 4 libri? Serve cambiare la **struttura** della tabella!

# 1FN: la soluzione

**Chiave primaria** sottolineata · *chiave esterna* in corsivo

- `Utenti(` **Tessera** `, Nome, Via, CAP, Città)` ← indirizzo **scomposto**
- `Telefoni(` **Tessera, Numero** `)` con *Tessera* → Utenti ← multivalore → **nuova tabella**
- `Prestiti(` **CodPrestito** `,` *Tessera* `,` *CodLibro* `, DataPrestito, DataRestituzione)`

> Ora il numero di libri in prestito è **illimitato**: basta **aggiungere righe**, non colonne. Bonus: possiamo memorizzare anche le date del prestito.

Regola pratica: **colonne numerate** (`Libro1, Libro2…`) o **liste separate da virgola** = 1FN violata.

# Seconda forma normale (2FN)

> Una relazione è in **2FN** se è in **1FN** e **ogni attributo non-chiave dipende dall'intera chiave**, non solo da una parte di essa.

- riguarda **solo** tabelle con **chiave composta**
- se la chiave è formata da **un solo attributo**, la tabella in 1FN è **automaticamente in 2FN**

Domanda da farsi per ogni attributo non-chiave:

> *"Per conoscere questo valore mi serve **tutta** la chiave, o ne basta un **pezzo**?"*

# 2FN: esempio (iscrizioni ai corsi pomeridiani)

`Iscrizioni(` **Matricola, CodCorso** `, NomeStudente, Classe, TitoloCorso, Ore, Voto)`

| Matricola | CodCorso | NomeStudente | Classe | TitoloCorso | Ore | Voto |
|---|---|---|---|---|---|---|
| 501 | C1 | Verdi | 5A | Python | 20 | 8 |
| 501 | C2 | Verdi | 5A | Robotica | 30 | 7 |
| 502 | C1 | Bruni | 5B | Python | 20 | 9 |

Dipendenze funzionali:

1. `{Matricola, CodCorso} → Voto` ✓ dipende da tutta la chiave
2. `Matricola → NomeStudente, Classe` ✗ parziale
3. `CodCorso → TitoloCorso, Ore` ✗ parziale

# 2FN: la scomposizione

Ogni dipendenza parziale diventa una **nuova tabella**, con il **determinante come chiave**:

- `Studenti(` **Matricola** `, NomeStudente, Classe)` ← dalla DF 2
- `Corsi(` **CodCorso** `, TitoloCorso, Ore)` ← dalla DF 3
- `Iscrizioni(` ***Matricola***, ***CodCorso*** `, Voto)` ← resta solo ciò che dipende da tutta la chiave

> Nella tabella `Iscrizioni` i due attributi della chiave diventano anche **chiavi esterne** verso `Studenti` e `Corsi`: è la classica tabella che traduce un'**associazione N:N** dell'E/R.

# Terza forma normale (3FN)

> Una relazione è in **3FN** se è in **2FN** e **ogni attributo non-chiave dipende direttamente dalla chiave**, cioè **non** esistono attributi non-chiave che dipendono da **altri attributi non-chiave**.

- elimina le **dipendenze transitive**
- può essere violata anche con **chiave semplice** (un solo attributo)

Domanda da farsi:

> *"Questo attributo descrive davvero la **chiave** della tabella, o descrive **un altro attributo**?"*

# 3FN: esempio (studenti e classi)

`Studenti(`**Matricola**`, Cognome, Classe, Aula, Coordinatore)`

| Matricola | Cognome | Classe | Aula | Coordinatore |
|---|---|---|---|---|
| 501 | Verdi | 5A | Lab 3 | Prof. Leoni |
| 502 | Bruni | 5B | A12 | Prof. Volpi |
| 503 | Neri | 5A | Lab 3 | Prof. Leoni |
| 504 | Galli | 5A | Lab 3 | Prof. Leoni |

- chiave semplice → la tabella è **sicuramente in 2FN**
- ma: `Matricola → Classe` e `Classe → Aula, Coordinatore`
- Aula e Coordinatore descrivono **la classe**, non lo studente → **dipendenza transitiva** ✗

Anomalia: cambia il coordinatore della 5A → modifica su **tutte** le righe degli studenti della 5A.

# 3FN: la scomposizione

- `Classi(` **Classe** `, Aula, Coordinatore)`
- `Studenti(` **Matricola** `, Cognome,` *Classe* `)`

| Matricola | Cognome | Classe |
|---|---|---|
| 501 | Verdi | 5A |
| 502 | Bruni | 5B |
| 503 | Neri | 5A |
| 504 | Galli | 5A |

| Classe | Aula | Coordinatore |
|---|---|---|
| 5A | Lab 3 | Prof. Leoni |
| 5B | A12 | Prof. Volpi |

Il determinante (`Classe`) **resta** in `Studenti` come **chiave esterna**: così, con un **JOIN**, ricostruisco la tabella originale.

# Il trucco per ricordarle

> Ogni attributo non-chiave deve dipendere
> **dalla chiave** (1FN), **da tutta la chiave** (2FN) e **da nient'altro che la chiave** (3FN).

| Se trovo… | Forma violata | Rimedio |
|---|---|---|
| liste, colonne numerate, attributi composti | 1FN | atomizzare / nuova tabella |
| attributo che dipende da **un pezzo** della chiave composta | 2FN | nuova tabella con quel pezzo come chiave |
| attributo che dipende da un **altro non-chiave** | 3FN | nuova tabella con quel non-chiave come chiave |

# L'algoritmo di scomposizione

1. **Individua** tutte le dipendenze funzionali e le chiavi candidate della tabella T
2. Per ogni dipendenza `X → Y` che **viola** una forma normale:
   a. crea una **nuova tabella** con gli attributi `X ∪ Y`
   b. il **determinante X** è la **chiave primaria** della nuova tabella
   c. dalla tabella di partenza **togli Y** e **lascia X** come **chiave esterna**
3. **Ripeti** il passo 2 finché nessuna dipendenza viola le forme normali

I passi 2 e 3 sono meccanici. Il passo 1 **no**: richiede di **capire la realtà** descritta. È lì che si sbaglia di più.

# Esempio completo · passo 1: le dipendenze

Riprendiamo `OrdiniFlat(NumOrdine, Data, CodCli, NomeCli, Città, CodProd, Descrizione, Prezzo, Qtà)`

Dipendenze funzionali (dalla realtà del negozio):

1. `{NumOrdine, CodProd} → Qtà`
2. `NumOrdine → Data, CodCli`
3. `CodCli → NomeCli, Città`
4. `CodProd → Descrizione, Prezzo`

Chiave: **{NumOrdine, CodProd}** (determina tutto, direttamente o transitivamente).

Diagnosi: è in 1FN · **non** è in 2FN (DF 2 e 4 sono **parziali**) · **non** è in 3FN (DF 3 è **transitiva**).

# Esempio completo · passi 2–3: scomposizione

**Violazione 2FN (DF 4)** → `Prodotti(` **CodProd** `, Descrizione, Prezzo)`

Resta: `R1(NumOrdine, CodProd, Data, CodCli, NomeCli, Città, Qtà)`

**Violazione 2FN (DF 2)** → porto via `NumOrdine` con tutto ciò che ne dipende (anche transitivamente):

`Ordini(` **NumOrdine** `, Data, CodCli, NomeCli, Città)`

Resta: `RigheOrdine(` **NumOrdine, CodProd** `, Qtà)` → **in 3FN ✓**

**Violazione 3FN in Ordini (DF 3)** → `Clienti(` **CodCli** `, NomeCli, Città)`

Resta: `Ordini(` **NumOrdine** `, Data,` *CodCli* `)` ✓

# Esempio completo · lo schema finale

- `Clienti(` **CodCli** `, NomeCli, Città)`
- `Prodotti(` **CodProd** `, Descrizione, Prezzo)`
- `Ordini(` **NumOrdine** `, Data,` *CodCli* `)`
- `RigheOrdine(` ***NumOrdine***, ***CodProd*** `, Qtà)`

Rivediamo le anomalie:

| Anomalia | Prima | Dopo |
|---|---|---|
| Rossi cambia città | 4 righe da modificare | **1 riga** in `Clienti` |
| Cancello ordine 1002 | perdo il cliente Bianchi | Bianchi **resta** in `Clienti` |
| Nuovo prodotto mai ordinato | impossibile | **1 INSERT** in `Prodotti` |

# Esempio completo · dallo schema all'SQL

```
CREATE TABLE Clienti (
  CodCli    CHAR(3)      PRIMARY KEY,
  NomeCli   VARCHAR(50)  NOT NULL,
  Citta     VARCHAR(40)
);
CREATE TABLE Prodotti (
  CodProd      CHAR(3)       PRIMARY KEY,
  Descrizione  VARCHAR(80)   NOT NULL,
  Prezzo       DECIMAL(8,2)  CHECK (Prezzo >= 0)
);
CREATE TABLE Ordini (
  NumOrdine  INT   PRIMARY KEY,
  Data       DATE  NOT NULL,
  CodCli     CHAR(3) NOT NULL REFERENCES Clienti(CodCli)
);
CREATE TABLE RigheOrdine (
  NumOrdine  INT      REFERENCES Ordini(NumOrdine),
  CodProd    CHAR(3)  REFERENCES Prodotti(CodProd),
  Qta        INT      CHECK (Qta > 0),
  PRIMARY KEY (NumOrdine, CodProd)
);
```

# Un dettaglio da progettista: il prezzo "storico"

Se a novembre il Mouse passa da 15 € a 18 €, aggiorno `Prodotti.Prezzo`…
…e **tutti gli ordini vecchi** risultano pagati 18 €! ✗

Il problema: il prezzo **pagato** non dipende solo dal prodotto, ma da **prodotto + ordine**.

Soluzione: `RigheOrdine(` **NumOrdine, CodProd** `, Qtà, PrezzoApplicato)`
In `Prodotti` resta il **prezzo di listino attuale**.

Non è ridondanza: sono **due informazioni diverse** (listino oggi ≠ prezzo pagato allora). Ancora una volta: **le DF le decide la realtà**.

# Scomposizione senza perdita di informazione

Normalizzare **non** significa cancellare dati: le nuove tabelle sono **proiezioni** di quella iniziale

`Clienti = π CodCli, NomeCli, Città (OrdiniFlat)`

e la tabella originale si **ricostruisce con i JOIN** sulle chiavi esterne:

```sql
SELECT O.NumOrdine, O.Data, C.CodCli, C.NomeCli, C.Citta,
       P.CodProd, P.Descrizione, P.Prezzo, R.Qta
FROM RigheOrdine R
  JOIN Ordini   O ON R.NumOrdine = O.NumOrdine
  JOIN Clienti  C ON O.CodCli    = C.CodCli
  JOIN Prodotti P ON R.CodProd   = P.CodProd;
```

Per questo l'algoritmo **lascia sempre il determinante** come chiave esterna.

# Oltre la 3FN: forma normale di Boyce-Codd (BCNF)

> Una relazione è in **BCNF** se, per ogni dipendenza `X → Y` non banale, **X è una superchiave**.

Esempio: `Lezioni(Studente, Materia, Docente)`

- ogni studente, per una materia, ha **un solo** docente: `{Studente, Materia} → Docente`
- ogni docente insegna **una sola** materia: `Docente → Materia`

Chiavi candidate: `{Studente, Materia}` e `{Studente, Docente}` → tutti gli attributi fanno parte di una chiave, quindi **nessun non-chiave**: la tabella è in **3FN**.
Ma `Docente` **non è superchiave** → **non è in BCNF** (la materia di un docente si ripete per ogni suo studente).

Scomposizione: `Docenti(`**Docente**`, Materia)` · `Assegnazioni(`**Studente**, ***Docente***`)`

# Il prezzo della normalizzazione

Più tabelle = interrogazioni **più complesse** (servono i JOIN).

Esempio: nome del reparto del dipendente con matricola 100.

**Prima** (tabella unica R)

$\pi$ NomeReparto ($\sigma$ Matricola=100 (R))

```sql
SELECT NomeReparto
FROM R
WHERE Matricola = 100;
```

**Dopo** (Dipendenti + Reparti)

$\pi$ NomeReparto ($\sigma$ Matricola=100 (Dipendenti $\bowtie$ Reparti))

```sql
SELECT Rp.NomeReparto
FROM Dipendenti D
  JOIN Reparti Rp
    ON D.CodReparto = Rp.CodReparto
WHERE D.Matricola = 100;
```

Ma **integrità e consistenza** dei dati valgono più di qualche JOIN in più.

# Quando la ridondanza è accettabile: denormalizzazione

## Database operativi (OLTP)

- dati **aggiornati di continuo** (ordini, prenotazioni, voti)
- rischio di inconsistenza alto
- ⇒ **normalizzare** (almeno 3FN)

## Database di analisi (data warehouse, OLAP)

- dati **in sola lettura**, caricati periodicamente
- servono **query veloci** su grandi volumi
- ⇒ ridondanza **tollerata o voluta** (denormalizzazione)

> Anche in un DB operativo si può denormalizzare **consapevolmente** un campo calcolato (es. `TotaleOrdine`), ma solo se si garantisce l'aggiornamento (trigger, procedure) e lo si **giustifica** nel progetto.

# Normalizzazione e seconda prova di maturità

Una traccia tipica chiede: **analisi** → **schema concettuale (E/R)** → **schema logico** → **SQL** → **query**.

Come usare la normalizzazione nell'elaborato:

- dopo lo schema logico, **dichiara** che le tabelle sono in **3FN** e **motiva** in 2–3 righe
- per ogni tabella con chiave composta, verifica che **nessun attributo dipenda da una parte** della chiave
- controlla che nessuna tabella contenga **descrizioni di un'altra entità** (nome cliente in Ordini, nome città in Utenti…)
- se lasci una ridondanza (es. un totale), **dichiaralo come scelta** e spiega perché

Possibili quesiti: "*Si illustrino le anomalie dovute alla ridondanza con un esempio*", "*Si porti in 3FN la seguente tabella…*".

# Checklist rapida

- ☐ Ogni cella contiene **un solo valore atomico**? → 1FN
- ☐ Niente **colonne numerate** o liste separate da virgole? → 1FN
- ☐ La chiave è **composta**? Ogni non-chiave dipende da **tutta** la chiave? → 2FN
- ☐ Qualche non-chiave **descrive un altro non-chiave** (un codice → il suo nome)? → 3FN
- ☐ Ogni tabella parla di **una sola "cosa"** (un'entità o un'associazione)?
- ☐ Ogni scomposizione ha lasciato una **chiave esterna** per il JOIN?

Segnale d'allarme classico: la coppia `CodXxx` + `NomeXxx` in una tabella che non è la tabella di Xxx.

# Esercizi

**E1 · Noleggio bici** — portare in 3FN:

`Noleggi(CodNoleggio, DataInizio, Ore, CodCliente, NomeCliente, Telefono, CodBici, Modello, TariffaOraria)`

(ogni bici ha un modello; la tariffa dipende dal modello)

**E2 · Biglietti per concerti** — portare in 3FN:

`Biglietti(CodEvento, NumPosto, NomeEvento, DataEvento, Locale, IndirizzoLocale, Acquirente)`

**E3 · Riconosci la forma normale** (chiave sottolineata):

a. `Voti(` **Matricola, Materia, Data** `, Voto, NomeStudente)`

b. `Libri(` **ISBN** `, Titolo, CodEditore, NomeEditore)`

c. `Squadre(` **CodSquadra** `, Nome, Giocatori)` con Giocatori = "Rossi, Neri, Galli"

# Soluzioni E1 ed E2

**E1** — chiave `CodNoleggio` (semplice → già 2FN). DF transitive:

`CodCliente → NomeCliente, Telefono` · `CodBici → Modello` · `Modello → TariffaOraria`

- `Clienti(` **CodCliente** `, NomeCliente, Telefono)`
- `Modelli(` **Modello** `, TariffaOraria)`
- `Bici(` **CodBici** `,` *Modello* `)`
- `Noleggi(` **CodNoleggio** `, DataInizio, Ore,` *CodCliente* `,` *CodBici* `)`

**E2** — chiave `{CodEvento, NumPosto}`.

`CodEvento → NomeEvento, DataEvento, Locale` (**parziale**) · `Locale → IndirizzoLocale` (**transitiva**)

- `Locali(` **Locale** `, IndirizzoLocale)`
- `Eventi(` **CodEvento** `, NomeEvento, DataEvento,` *Locale* `)`
- `Biglietti(` ***CodEvento***, **NumPosto** `, Acquirente)`

# Soluzioni E3

| Tabella | Forma normale | Motivo | Correzione |
|---|---|---|---|
| a. `Voti` | **1FN** (non 2FN) | `Matricola → NomeStudente`: dipendenza **parziale** | `Studenti(Matricola, NomeStudente)` + `Voti(Matricola, Materia, Data, Voto)` |
| b. `Libri` | **2FN** (non 3FN) | `CodEditore → NomeEditore`: dipendenza **transitiva** | `Editori(CodEditore, NomeEditore)` + `Libri(ISBN, Titolo, CodEditore)` |
| c. `Squadre` | **non in 1FN** | `Giocatori` è **multivalore** | `Giocatori(CodGiocatore, Cognome, CodSquadra)` |

# Riepilogo

**Ridondanza** → anomalie di **inserimento, modifica, cancellazione**

**Dipendenze funzionali**: si ricavano dalla **realtà**

**1FN** valori atomici · **2FN** niente dipendenze parziali · **3FN** niente dipendenze transitive

**Scomporre** senza perdere dati: il determinante resta come **chiave esterna**

Normalizzare nei DB **operativi**, denormalizzare (con criterio) in quelli di **analisi**